

Application Module^X User Guide

**Release 300
AX09-300**

Application Module^x

**Application Module^x
User Guide**

**Release 300
AX09-300
12/98**

Notices and Trademarks

© Copyright 1998 by Honeywell Inc.

Revision 01 – December 1, 1998

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

Honeywell, **TotalPlant**, and Universal Station^x are U.S. registered trademarks of Honeywell Inc.

Other brand or product names are trademarks of their respective owners

Honeywell
Industrial Automation and Control
Automation College
2820 West Kelton Lane
Phoenix, AZ 85053-3028

1-800-852-3211

About This Publication

This publication is intended for Application Module^x users—application developers and administrators. Application Module^x is part of Honeywell's TotalPlant Solution (TPS) System. TPS is the evolution of TDC 3000X.

This publication includes an overview of the Application Module^x and describes its function in the Honeywell **TotalPlant** environment. It addresses the security features of the Application Module^x, and includes an introduction and example of the use of OpenDDA functionality to access LCN data from an application.

This publication is not intended to serve as a reference manual. The application developer should refer to the OpenDDA publications for detailed reference material. The system administrator should refer to the *Application Module^x System Administration* manual for specific system administration procedures, and the maintenance person should refer to *Application Module^x Service* and *Application Module^x Troubleshooting* for reference information.

This publication supports A^xM Release 300 and TPS system software releases 433, 531, 600 and later.

Table of Contents

1. INTRODUCTION.....	13
1.1 Overview	13
What is the Application Module ^x ?	13
The control and information domains	14
Software	14
Open yet secure	14
X Windows environment.....	15
1.2 Application Module^x Architecture.....	16
Chassis types	16
Five-Slot Module configuration	16
Ten-Slot Module configuration.....	16
Function of components	17
Coprocessor hardware	18
Coprocessor software	18
1.3 Application Module^x System Configurations	19
Introduction.....	19
Standalone coprocessor.....	19
Minimum network configuration with U ^x S	20
Minimum network configuration with Hewlett-Packard workstation	21
Expanded network configuration	22
Initial setup and maintenance configuration	23
Where to find more information	24
2. SECURITY	25
2.1 HP-UX Security.....	25
Introduction.....	25
Some important categories.....	25
File permissions.....	25
Network security	26
User security.....	26
Audit Files.....	26
2.2 Physical and Procedural Security	27
Physical security.....	27
Procedural security.....	27
2.3 LCN Security.....	28
Overview.....	28
Topics to be covered	28
A ^x M failover to AM	28
Directory for CL-initiated applications	29
Using test data with OpenDDA	29
The global X-access switch	30
Sample session with xaccess	31
Function of XACCES	31

Table of Contents

3. FILE SYSTEM	33
3.1 Overview.....	33
Discussion	33
3.2 Directories.....	34
Summary of key directories	34
3.3 Files.....	35
Configuration files discussion.....	35
Reference configuration files.....	35
Working configuration files	36
Data files	36
Executables.....	36
Libraries	37
Modified HP-UX files	38
4. PERFORMANCE MEASUREMENT	39
4.1 Overview.....	39
Introduction	39
Accessing the parameters from the LCN side	39
Accessing the parameters from the X side.....	39
AXMPERF schematic	40
DATACHNG schematic	40
Custom schematic	40
CL access.....	40
Programming example	41
4.2 \$RCVSTA(n) Parameter.....	42
Discussion	42
4.3 \$RCVCNT(n) Parameter	44
Discussion	44
4.4 \$XMITSTA(n) Parameter.....	45
Discussion	45
4.5 \$XMITCNT(n) Parameter	46
Discussion	46
5. HIBERNATION.....	47
5.1 Overview.....	47
What is hibernation?	47
What will be covered	47
Advantages of hibernation.....	47
Event relationships.....	48
5.2 CL Support.....	49
Supporting functions.....	49
Initiate_Task.....	49
Activate_Task.....	50
Terminate_Task	50
Get_Hiber_Task_Status	51

Table of Contents

5.3	OpenDDA Support	52
	Supporting function.....	52
	evt_data.....	52
	Reference	52
5.4	X-Side Support Tools.....	53
	Introduction	53
	display_appls	53
	kill_appls.....	54
6.	CONFIGURATION OF DATA ACCESS PRIORITY	55
6.1	Introduction	55
	A*M services	55
	Application class.....	55
	High priority control applications.....	55
	Low priority information level applications	55
	Configuration goals.....	56
	Resources	56
	Data access channels.....	56
	Data access servers	57
	Summary	57
6.2	xdaconfig Configuration Tool.....	58
	Files.....	58
	Configuration tool	58
	Discussion	58
	Sample session	59

Tables and Figures

Table 1-1	A ^x M Five-Slot Module Configuration	16
Table 1-2	A ^x M Ten-Slot Module Configuration.....	16
Table 1-3	Function of A ^x M Components	17
Table 3-1	Key Directories.....	34
Table 3-2	Reference Configuration Files.....	35
Table 3-3	Working Configuration Files	36
Table 3-4	Data Files	36
Table 3-5	Executable Files.....	37
Table 3-6	Library Files.....	37
Table 3-7	Modified HP-UX Files.....	38
Table 4-1	Using the AXMPERF Schematic.....	40
Table 4-2	Using the DATACHNG Schematic.....	40
Table 4-3	Elements of the \$RCVSTA(n) Array	42
Table 4-4	Elements of the \$RCVCNT(n) Array	44
Table 4-5	Elements of the \$XMITSTA(n) Array	45
Table 4-6	Elements of the \$XMITCNT(n) Array	46
Table 5-1	Event Data Structure.....	52
Figure 1-1	A ^x M Block Diagram.....	13
Figure 1-2	Standalone Coprocessor Configuration.....	19
Figure 1-3	Minimum Network Configuration with a U ^x S	20
Figure 1-4	Minimum Network Configuration with an HP workstation	21
Figure 1-5	Initial Setup and Maintenance Configuration	23
Figure 3-1	Base File System Structure	33
Figure 5-1	Relationship of CL Events and X-side Hibernation.....	48

Acronyms

A ^x M	Application Module ^x (Application Module with Extensions)
DAT	Digital Audio Tape
LAN	Local Area Network
LCN	Local Control Network
OpenDDA	(Open) Data Definition and Access
PCDE	PC Data Exchange
PIN	Plant Information Network
SAM	System Administration Manager
TCP/IP	Transmission Control Protocol/Internet Protocol
TDC	Total Distributed Control
TPH	Total Plant History
UCN	Universal Control Network
US	Universal Station
U ^x S	Universal Station ^x (Universal Station with Extensions)
WSI	Workstation Interface

References

Publication Title	Publication Number	Binder Title	Binder Number
Application Module ^x and OpenDDA Specification and Technical Data	AX03-300	Application Module ^x	2094/3094
Application Module ^x Customer Release Guide	AX04-300	Application Module ^x	2094/3094
Application Module ^x System Administration	AX11-300	Application Module ^x	2094/3094
Application Module ^x Service	AX13-410/510/610	Application Module ^x	2094/3094
Application Module ^x Troubleshooting	AX13-300	Application Module ^x	2094/3094
License Installation and Administration A ^x M	LA20-300	Application Module ^x	680/2094/3094
Five/Ten-Slot Module Service	LC13-400/500	LCN Service - 1	2060-2/3060-2
LCN System Installation	SW20-500	LCN Installation	2025/3025
OpenDDA Customer Release Guide	DD04-300	OpenDDA	680
OpenDDA User's Guide	DD11-300	OpenDDA	680
OpenDDA Reference Manual	DD27-300	OpenDDA	680

Hewlett-Packard Documentation on CD-ROM

Hewlett-Packard Documentation at <http://www.docs.hp.com>

1. Introduction

1.1 Overview

What is the Application Module^x?

The Application Module^x (A^xM) is a major component of Honeywell's **TotalPlant** Solution (TPS) system. It uses a dual-processor architecture—a traditional Application Module (AM) combined with a powerful workstation processor, which we will refer to as the coprocessor. The Application Module provides reliability, security, and a link to the TPS that may already exist at the customer's plant, preserving investment in process control hardware and software. The coprocessor uses industry-standard, state-of-the-art workstation hardware and software, providing an open environment for development and execution of advanced **process control** programs and for acquisition of data for higher-level **production control** functions. The coprocessor provides a link to the Plant Information Network—an Ethernet LAN that can include PCs, workstations, and mainframes, all contributing to the overall operation of the plant. Figure 1-1 shows a block diagram of the A^xM.

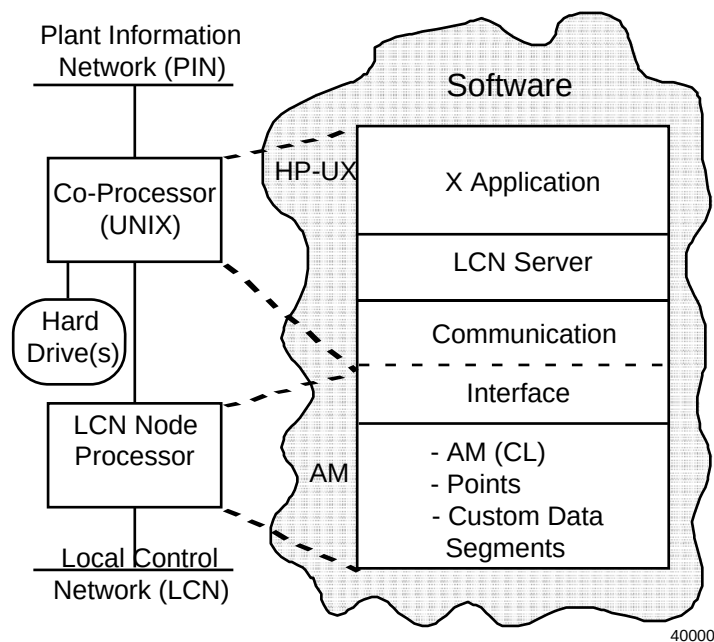


Figure 1-1 A^xM Block Diagram

The control and information domains

The collection of hardware and software that is most intimately involved in the control of the process is referred to as the **control domain**. This includes the traditional LCN and its nodes and control programs, the UCN and Data Hiway and their controllers, and the field instrumentation infrastructure—the sensors, valves, and other control elements.

The **information domain** includes the various processors connected to the Plant Information Network (PIN)—PC, Macintosh, VAX, and mainframe platforms, as well as U^xS and A^xM coprocessors. These computers use data generated by the control domain for production management functions such as unit cost and quality measurement, inventory control and inventory cycle time analysis, scheduling, process optimization, and other production management functions.

Software

The software “glue” that connects the control and information domains allowing the open environment includes CL/AM enhancements, security features, coprocessor development resources, and the standard **OpenDDA** (Data Definition and Access). **OpenDDA** is the support software that allows a coprocessor application program to access LCN data.

Open yet secure

In order to provide an environment that is both open and secure, Honeywell has carefully chosen the areas to open. By embracing the Fieldbus standard, Honeywell has opened up the instrumentation level so as to be compatible with industry standard devices from a large number of vendors. By adding a standard workstation with PIN interface as a coprocessor in the US and AM (thereby creating the U^xS and A^xM), Honeywell has provided users and third-party developers with a means to develop powerful applications using standard languages, that run on standard platforms, and that can use live process data.

While the field instrumentation level and the information domain level are now open, the LCN and its process controllers are retained so as to continue the security and reliability that are the hallmark of the TPS System. The “view of the process” and the “path to the valve” are not compromised in this open environment.

X Windows environment

Computers on the PIN can use the X Windows Client/Server environment. This requires:

- A 5-10 Mbit Ethernet or Token Ring Local Area Network (the PIN)

- TCP/IP and X11.R5 (or later) software in both clients and servers

- Motif window manager software in all clients

The power and versatility of the X Windows environment allows a user on a PIN device of a given platform (for example, a PC configured as an X-Server) to open a window and run an application in another PIN device that is of a different platform (for example, an A^xM whose coprocessor is running HP-UX).

In X Windows convention, the client is the device running the application and the device providing the screen, keyboard, and mouse services is the server.

1.2 Application Module^x Architecture

Chassis types

The A^xM is available in three chassis configurations:

Five-Slot Module—uses the K4LCN node processor, which has hardware floating point support (R500 and R600)

Five-Slot Module—uses the K2LCN node processor, and therefore does not support hardware floating point

Ten-Slot Module—uses the HMPU, which has hardware floating point support

Five-Slot Module configuration

The following table shows the configuration of the Five-Slot Module versions of the A^xM.

Table 1-1 A^xM Five-Slot Module Configuration

Slot	Front	Rear
5	Optional Application Board	Optional I/O board
4	Hard Disk Drive Tray	
3	(occupies two slots)	HDDT I/O
2	WSI2	WSI2 I/O
1	K2LCN/K4LCN	LCN I/O

Ten-Slot Module configuration

The following table shows the configuration of the Ten-Slot Module version of the A^xM.

Table 1-2 A^xM Ten-Slot Module Configuration

Slot	Front	Rear
10	Unusable slot	
9	Unusable slot	
8	Hard Disk Drive Tray	
7	(occupies two slots)	HDDT I/O
6	WSI2	WSI2 I/O
5	Optional Application Board	Optional I/O board
4	Optional memory board	
3	QMEM-4	
2	LLCN	LCN I/O
1	HMPU	

Function of components

The following table lists the function of and options for each of the components listed in Table 1-1 and Table 1-2.

Table 1-3 Function of A^xM Components

Component	Function
K4LCN	Node processor used in Five-Slot Module A ^x M (R500 and R600). Available with the following memory sizes: 4 Mw (Note: A ^x M R300 requires a minimum of 6 Mw) 8 Mw 16 Mw Has hardware floating point support.
K2LCN	Node processor used in Five-Slot Module A ^x M. Available with the following memory sizes: 4 Mw (Note: A ^x M R300 requires a minimum of 6 Mw) 8 Mw Does not have hardware floating point support.
HMPU	Node processor used in Ten-Slot Module A ^x M. Has 2 Mw of memory. Has hardware floating point support.
LLCN	LCN interface board (used only with HMPU version—this function is built into the K2LCN and K4LCN)
LCN I/O	I/O paddleboard that provides connection to the LCN A and LCN B cables.
QMEM-4	Memory board—provides 4 Mw additional memory. Used only with HMPU version
WSI2	Workstation Interface board. Contains the coprocessor board. Available with the following coprocessor memory options: 32 MB 64 MB 128 MB 256 MB Available with the following coprocessor speed options: 64 MHz 100 MHz
WSI2 I/O	I/O paddleboard associated with the WSI2 board. Provides interface to the PIN and to a console terminal or modem.
Hard Disk Drive Tray	Holds a primary hard disk drive and an optional hard disk drive. Each drive can be either: 525 MB 1 GB 2 GB 4 GB (Note: A ^x M 300 will only be delivered with 4 GB.) This tray occupies two card slot positions.
HDDT I/O	I/O paddleboard associated with the hard disk drive tray. Contains an optional SCSI interface connector that is not used with A ^x M and that must have a cable terminator installed.

Coprocessor hardware

The coprocessor (a Hewlett-Packard Model 743 processor board) is mounted on the WSI2 board (this board is also used in the Release 200 Universal Station X.) The board is based on the Hewlett-Packard PA-RISC (Precision Architecture-Reduced Instruction Set Computing) chip. The coprocessor is available in 64 MHz and 100 MHz versions, and with 32 MB, 64 MB, 128 MB, or 256 MB of memory.

Coprocessor software

The coprocessor operating system is version 10.20 of HP-UX. HP-UX is based on UNIX System V. It includes many of the most popular extensions from the University of California, Berkley, version of UNIX, and it provides full or partial compliance to numerous industry and international standards. For more information about the HP-UX operating system, refer to the Hewlett-Packard CD-ROM documentation.

1.3 Application Module^x System Configurations

Introduction

The Application Module^x can be used in a variety of ways. We will cover a few configurations that we anticipate will be typical usage scenarios, but they are not intended to represent all possible configurations.

Standalone coprocessor

The standalone coprocessor configuration shown in Figure 1-2 is the simplest configuration. Its characteristics are:

It is shipped with one or more applications preloaded and ready to run.

There is no Ethernet LAN (PIN) connection; therefore, there is no access to external media, which impacts as follows:

Data generated by an application cannot be used by other devices

You cannot backup the hard disk

You cannot load optional software or load software upgrades

If the hard disk fails, it must be replaced with a new drive that has the operating system and applications preloaded by the application supplier unless you have the optional DAT drive and the necessary tapes.

From a network security standpoint, this is the most secure configuration because there is no PIN connection.

From a US or U^xS on the LCN, you can activate CL that uses the AMCL06_2 extension set to initiate A^xM applications.

Note: If the A^xM is equipped with the optional DAT drive and a console device, you can backup the disk and load software.

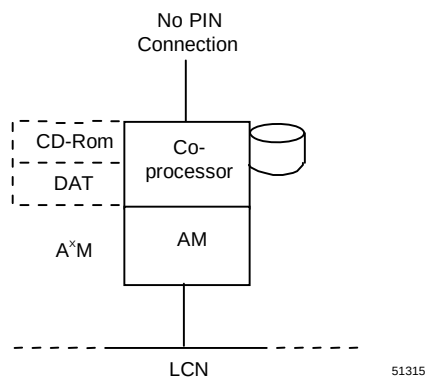


Figure 1-2 Standalone Coprocessor Configuration

Minimum network configuration with U^xS

If applications will be developed locally, or if access to a removable media drive is required for backups, software installation, or other system administration tasks, then a minimum of one other network-resident device is required. This can be a U^xS, as shown in Figure 1-3. The characteristics are:

The A^xM and U^xS are the only terminals on a point-to-point local LAN.

The absence of other devices on the LAN makes this configuration secure from the standpoint of network security.

You can perform system administration tasks from the U^xS by doing a remote login (`rlogin`) to the A^xM over the network and using SAM—a system administration manager utility supplied with HP-UX.

The Digital Audio Tape (DAT) drive on the U^xS can be used for hard drive backups and for software installations and upgrades.

You can develop A^xM applications from a window at the U^xS while logged into the A^xM.

You can use the U^xS as the console terminal of the A^xM for initial network configuration and for maintenance. Connect the U^xS serial printer port to the A^xM console port with a null modem cable, and use terminal emulation software on the U^xS (Kermit software is supplied with the U^xS).

From the U^xS (or from any US or U^xS on the LCN), you can activate CL that uses the AMCL06_2 extension set to initiate A^xM applications.

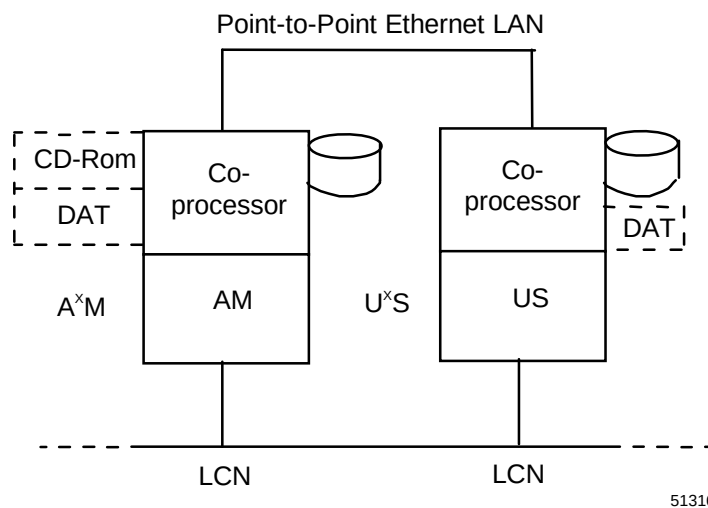


Figure 1-3 Minimum Network Configuration with a U^xS

Minimum network configuration with Hewlett-Packard workstation

This configuration is similar to the previous configuration except that the U^xS is replaced by an HP Workstation. The A^xM and HP workstation are the only terminals on a point-to-point local Ethernet LAN. The characteristics are:

The absence of other devices on the LAN makes this configuration secure from the standpoint of network security.

You can perform system administration tasks from the workstation by doing a remote login (rlogin) to the A^xM over the network and using SAM—a system administration manager utility supplied with HP-UX.

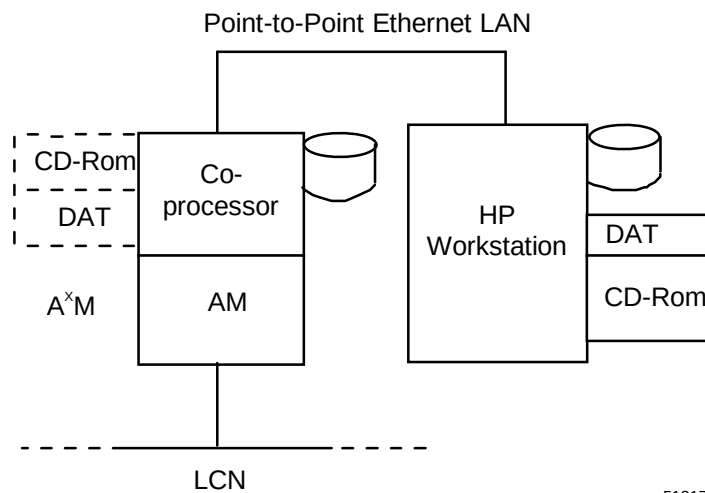
The Digital Audio Tape (DAT) drive on the workstation can be used for hard drive backups and for software installations and upgrades.

You can develop A^xM applications from a window at the workstation while logged into the A^xM.

The workstation can be used as the console terminal of the A^xM for initial network configuration and for maintenance. Connect the HP workstation serial port to the A^xM console port with a null modem cable and use terminal emulation software.

From a US or U^xS on the LCN, you can activate CL that uses the AMCL06_2 extension set to initiate A^xM applications.

The HP workstation provides access to the HP CD-ROM documentation.



51317

Figure 1-4 Minimum Network Configuration with an HP workstation

Expanded network configuration

The preceding configurations are characterized by minimum Ethernet LAN networks. In the standalone configuration, the A^xM is not connected to a LAN. In the minimum configurations with an HP workstation or an U^xS, a single point-to-point LAN is used between the A^xM and the development/administration station. These are the simplest configurations, and the most secure with respect to network access.

At the other extreme, the A^xM can be connected to a complex Ethernet LAN that functions as the Plant Information Network. In this environment, the A^xM can coexist on the PIN with multiple devices and platforms, such as U^xS, workstation, VAX, PC, Macintosh, and mainframe. With appropriate software and configuration, these diverse devices can utilize and exchange LCN data and use this data in plant and process management applications.

Initial setup and maintenance configuration

The A^xM is shipped with networking enabled, but configured for maximum security. Permissions for all services (in /var/adm/inetd.sec) are set to “deny.” Root is the only user allowed to log in, and this is allowed only from the local console terminal. If the A^xM is to be connected to the Ethernet LAN, you must connect a local console terminal and then log in as root and perform the necessary configuration changes. After the initial configuration, authorized users can log in from the network and perform system administration and development activities.

The console terminal connection may also be required for system maintenance. If the network is down, the console is the only device from which you can access HP-UX. The console is also the only device from which you can observe the HP-UX boot process.

The console device can be a VT100 or a device using VT100 emulation. Honeywell recommends and supports the use of a U^xS or the HP workstation. Either of these devices can be connected to the A^xM console port with a serial null modem communications cable. Both are shipped with Kermit software, which can provide the VT100 emulation.

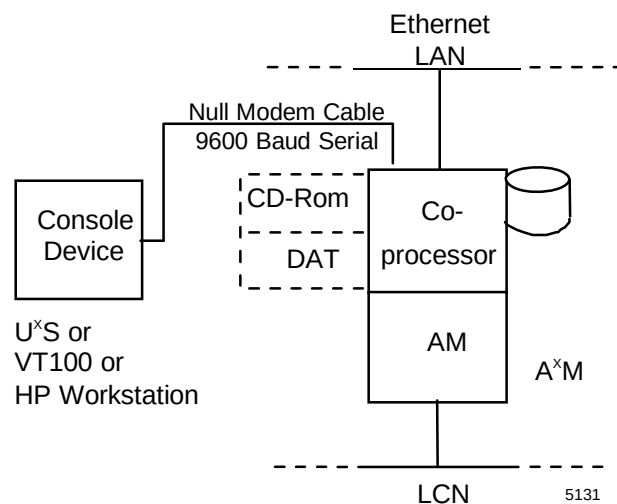


Figure 1-5 Initial Setup and Maintenance Configuration

Where to find more information

The *Application Module^x Service* manual contains instructions on connecting the VT100-like terminal, U^xS, or HP Workstation to the console port of the A^xM. The *Application Module^x System Administration* manual contains the procedures to invoke Kermit and perform the required A^xM configuration.

2. Security

2.1 HP-UX Security

Introduction

UNIX has long been touted for its security features. HP-UX is a UNIX derivative and retains these security features. Part of this security derives from the operating system architecture, which is designed to prevent any program from corrupting any other program or the operating system itself.

Some important categories

Some of the most important security categories of HP-UX are:

- File permissions

- Network security

- User security

- Audit files

These and other categories are covered in *HP-UX System Security*, which is available in the HP CD-ROM documentation.

File permissions

Each file and directory can be set to allow or deny read, write, and execute permission for each of the following:

- Owner

- Group

- Others

These permissions can be changed with the *chmod* (change file mode) command. The owner and group can be changed with the *chown* and *chgrp* commands, respectively.

Another command, *chacl* (change access control list), extends the capabilities of the *chmod* command by allowing the user to allow or deny file access to additional users and/or groups. Up to 13 additional sets of permissions (called optional access control list entries) can be stored in the access control list of the file.

Refer to the man pages and/or the HP documentation for detailed information on the *chmod*, *chown*, *chgrp*, and *chacl* commands.

Network security

The file `/var/adm/inetd.sec` permits you to allow or deny specific services to individual users or groups of users. Examples of services are login, rlogin, telnet, and ftp. Workstations are normally shipped with all of these permissions set to “allow”—minimum security. The A^xM, on the other hand, is shipped with all permissions set to “deny” for maximum security. You, the user, can relax this security and permit whatever level of access is appropriate for your A^xM.

User security

HP-UX includes the traditional UNIX security features that are part of the user management process. Each user has a login name, user ID, group ID, and a password that is stored only in encrypted form. Passwords are not stored anywhere in the system in plain text (unencrypted form). They are encrypted using a highly secure technique. When a user logs in and enters a password, it is encrypted and compared with the encrypted version stored in the system.

In high-security situations, it is important to physically secure data transmission media between the user’s terminal and the A^xM to prevent interception of login sequences, which include the password in plain text. It is also important to educate users on the types of passwords to use, and to not use and to establish procedures that require the users to change passwords periodically. Each password can be set up with an aging factor—the number of days that it remains valid. Before the period ends, the user must change to a new password.

Audit Files

HP-UX provides the capability to audit computer use, both on an individual and system-wide basis. You can configure the HP-UX system to audit users, events, and system calls. You can use the System Administration Manager utility (SAM) to set up the auditing functions. Refer to the Hewlett-Packard CD-ROM manual *HP-UX System Security* for additional information about audit files.

2.2 Physical and Procedural Security

Physical security

Physical security involves protecting the system from damage or corruption by human and environmental factors. Some examples of physical security measures are:

- Fences, intrusion alarms, security guards

- Key locks, card access systems

- Fire alarms, sprinkler systems, automatic fire suppression systems.

- Secure off-site storage for backups, checkpoints, etc.

- Uninterruptible power supplies

Procedural security

You should define your security objectives and requirements and develop a management-approved security policy. The policy should determine a set of practices and procedures that are distributed and followed meticulously. Some examples of security practices and procedures are:

- A procedure and schedule for backups and checkpoints

- Regulation and monitoring of login access and access to critical files and directories

- A policy and schedule for changing passwords

- A policy requiring users to log off or use lock command when not at the terminal

- A policy and procedure regarding audit use, review, and analysis

2.3 LCN Security

Overview

The **TotalPlant** Solution (TPS) system has a well-deserved reputation for reliability. Security features are incorporated in the architecture and design of the A^xM hardware and software in order to preserve this reliability. These features are designed to protect the LCN control environment from:

- X-side failures
- Intrusion by hackers or unauthorized users on the network
- Corruption while testing new software that is not fully debugged
- Human error

Topics to be covered

The security features of the A^xM environment that will be covered are:

- A^xM failover to AM
- Special directory for CL-initiated applications
- OpenDDA development using test data
- The global X-access switch
- The AMCL06\$Store_XAccess call
- The XACCES external load module

A^xM failover to AM

If an X-side failure occurs, the AM will continue to function as follows:

The node status will go to WARNING.

Page 2 of the Status Detail Display will display the message

```
WS Interface (OK -> WARNING):Work Station Interface In Process -  
Please wait
```

CL applications that do not communicate with the X-side will continue to function normally. These will be applications that do not use the calls in the CL extension set AMCL06_2.

CL applications that do communicate with the X-side using calls from the AMCL06_2 set will get return statuses indicating errors. The specific actions that take place when this occurs are handled in the CL application, and therefore depend on how the application was coded.

Directory for CL-initiated applications

Each CL-initiated application must have its executable file, or a symbolic link to its executable file, in the directory `/users/axm`. It cannot be started from CL if it is in any other directory. This directory is owned by the user “axm” and the group associated with this directory is the group “axm.” The permissions for this directory, as shipped, are:

<u>Owner</u>	<u>Group</u>	<u>Other</u>
rwX	rwX	---

Application developers, when they are added to the system as users, will normally be assigned to the group “axm.” Therefore, when a developer has compiled and linked an application program, the permissions will allow the developer to store the executable, or create a symbolic link to the executable, in `/users/axm`. OpenDDA has an “Install” command option that will automatically create the symbolic link.

If you wish to tighten security even further, you can modify the permissions on `/users/axm` accordingly. For example, you can make superuser the only user with write permission.

Using test data with OpenDDA

OpenDDA (Data Definition and Access) provides the software tools that allow an application to access LCN data (to read data and, with security set properly, to write data.) An application that uses OpenDDA can be compiled with test data imbedded in the program. The application can be debugged off-line using this test data instead of on-line with live LCN data. The application can actually be developed and tested in a U^xS or a workstation. Only the A^xM, however, can access live data.

The global X-access switch

The global X-access switch controls when an X-side application can write LCN data. It is implemented by a Processor Status Data Point (PSDP) parameter called `$XACCESS` which has three possible states:

READONLY—The X-side can read but cannot write LCN data (default)

RW_LCN_I—The X-side can read LCN data, but can write LCN data only from applications that are initiated by CL on the LCN-side

READWRIT—The X-side can read and write LCN data from CL-initiated and non-CL-initiated applications (this state cannot be set by any means unless the external load module **XACCES** is loaded)

The state of the global X-access switch can be changed:

From the keyboard with Engineer access

From an LCN-side CL program (using a call available in `AMCL06_2`—the global X-access switch cannot be changed by a normal CL write to a parameter).

An X-side application cannot change the global X-access switch; however, a configuration file **xaccess.cfg** on the X-side provides the restart value for the switch. The **xaccess.cfg** file is modifiable by the tool **xaccess**, if the user has execute permission on the tool and read/write permission on the configuration file (root only in both cases as shipped). The configuration file **xaccess.cfg** is in the directory `/opt/TDC_Open/common/newconfig` and the tool **xaccess** is in the directory `/opt/TDC_Open/common/bin`.

In all cases, the **READWRIT** state cannot be set unless the **XACCES** external load module is loaded in the AM-side.

Sample session with xaccess

The following is a sample session running the tool xaccess.

```
cd /opt/TDC_Open/common/bin
```

```
xaccess
```

```
XACCESS: Configuration::Read/writes by all applications allowed
```

```
0. Read only
```

```
1. Read/writes by LCN initiated applications allowed
```

```
2. Read/writes by all applications allowed
```

```
Enter the new value for X to LCN access (0, 1, or 2): 2
```

Attention

\$XACCESS is not a checkpointed parameter. When the A*M node personality is loaded, the restart state of \$XACCESS is determined solely by a value encrypted in the X-side file xaccess.cfg. (The X-side must be running for the A*M node personality to load.) If the X-side is restarted while the node personality is running, state of \$XACCESS is not affected—it will remain in its current state.

Function of XACCES

The XACCES external load module, by its presence or absence, allows or disallows writes of LCN data from non-CL-initiated X-side applications. This is accomplished as follows:

If XACCES is loaded, the READWRIT state of the global X-access switch can be set by an Engineer from the keyboard, or by a CL call. When the READWRIT state is set, non-CL-initiated applications are allowed to write LCN data.

If XACCES is not loaded, the READWRIT state cannot be set, and therefore non-CL-initiated applications cannot write LCN data.

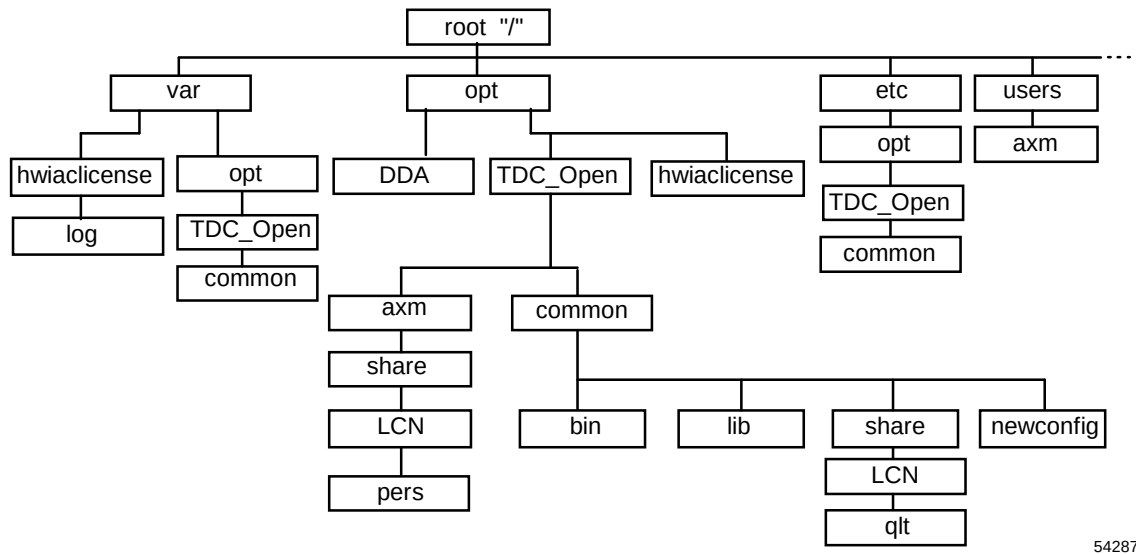
3. File System

3.1 Overview

Discussion

This section presents an overview of the base file structure of the A^xM. It is not intended to be a complete listing of all files and directories in the system. Its purpose is to identify the directories and files that are of special interest in the A^xM.

Figure 3-1 shows the base file system structure.



54287

Figure 3-1 Base File System Structure

3.2 Directories

Summary of key directories

Table 3-1 summarizes the key directories and lists the basic types of files in each. The files indicated in the table are explained in greater detail in subsequent tables.

Table 3-1 Key Directories

Directory	Contents
/var/opt/TDC_Open/common	Temporary and log files—includes the CDS and LCN daemon log files
/opt/TDC_Open/common/bin	Executable files cds_hdr, cdsdaemon, lcndaemon*, xaccess, display_appls, kill_appls, and xdaconfig
/opt/TDC_Open/common/lib	Libraries libcds.sl, libhiber.sl, and liblxs.sl.
/opt/TDC_Open/common/newconfig	Contains reference copies of the base configuration files and scripts that are used at system startup
/opt/TDC_Open/common/share	Include files cds.h and cds_err.h, axm_common.h and hiber.h
/opt/TDC_Open/common/share/LCN	Boot file boot_stub.bo
/opt/TDC_Open/axm/share/LCN/pers	Personality files
/opt/TDC_Open/common/share/LCN/qlt	QLT files—files used for test
/etc/opt/TDC_Open/common	Contains the working copies of the base configuration files and scripts that are used at system startup
/etc	Scripts that run at startup
/users/axm	Executables, or symbolic links to executables, for all CL-initiated applications
/opt/DDA	See OpenDDA User's Guide, subsection 4.1
/opt/hwiaclicense	See License Installation and Administration A ^x M
/var/hwiaclicense/log	See License Installation and Administration A ^x M

3.3 Files

Configuration files discussion

Certain key configuration files (reference copies) are stored in the directory */opt/TDC_Open/common/newconfig*. Working copies are placed in */etc/opt/TDC_Open/common* where they are actually used during startup. If the user modifies any of the working files, it is easy to go back to the original (reference) version.

Also, if Honeywell updates software, changes are placed only in the reference copies to avoid overwriting the working files that the user may have customized. Therefore, after an update, the user should check the files in */opt/TDC_Open/common/newconfig* to see if Honeywell has made any changes, and should incorporate these changes in the working copies in */opt/TDC_Open/common*.

Reference configuration files

The files in Table 3-2 reside in the */opt/TDC_Open/common/newconfig* directory.

Table 3-2 Reference Configuration Files

File	Purpose
personality.config	Indicates that the device is an A ^x M
personality.config.axm	Identifies personality files type and location
personality.config.axmk (R500 & R600)	Identifies K4LCN personality files type and location
rc	Reference copy of a script that is executed on startup
tdc3krc	Reference script that starts the proper daemon based on whether the device is an A ^x M or a U ^x S and on what type of coprocessor board it uses
xaccess.cfg	Configuration file that defines the as-shipped restart state of the Global X-access Switch (READONLY)
xdapriority.cfg	Configuration file that defines the as-shipped number of communication channels and Data Access servers reserved for high priority requests.

Working configuration files

The files in Table 3-3 reside in the */etc/opt/TDC_Open/common* directory.

Table 3-3 Working Configuration Files

File	Purpose
personality.config	This is a link to a file that contains the personality files location and type (U ^x S or A ^x M)
tdc3krc	Working copy of a script that starts the proper daemon based on whether the device is an A ^x M or a U ^x S and on what type of coprocessor board it uses—normally a copy of the reference script in <i>/opt/TDC_Open/common/newconfig</i> unless user has made changes to the working copy
xaccess.cfg	Configuration file that defines the current restart state of the Global X-access Switch—the utility program <i>xaccess</i> modifies this file
xdapriority.cfg	Configuration file that defines the current number of communication channels and Data Access servers reserved for high priority requests. The utility program <i>xdaconfig</i> modifies this file.

Data files**Table 3-4 Data Files**

File	Purpose
axmh*.*	All of the files that are required to load A ^x M 68020 processor based personality (resides in <i>/opt/TDC_Open/axm/share/LCN/pers</i>)
axmk*.*(R500 and R600)	Files to load A ^x M 68040 processor based personality (resides in <i>/opt/TDC_Open/axm/share/LCN/pers</i>)
qlt1.pi	Quality Logic Texts (QLTs) that are run on all boards when the node software is loaded (resides in <i>/opt/TDC_Open/common/share/LCN/qlt</i>)
boot_stub.bo	Boot file that is used for U ^x S and A ^x M (resides in <i>/opt/TDC_Open/common/share/LCN</i>)

Executables

All of these executables reside in the */etc/opt/TDC_Open/common/bin* directory.

Table 3-5 Executable Files

File	Purpose
cds_hdr	A utility for Honeywell use only
cdsdaemon	Handles the initiation from CL of X-side applications, and handles other functions for Honeywell software
lcndaemon*	This is a message handler daemon that performs the interface between the LCN node processor and the HP-UX coprocessor (there are three versions—the 743 version is used by the A*M)
xaccess	A utility that allows the HP-UX root user to change the restart state of the global X-access switch (refer to the topic “The global X-access switch” in Section 2)
display_appls	An X-side utility that allows the user to display information about CL-initiated X-side applications.
kill_appls	An X-side utility that allows the user to kill CL-initiated X-side applications, including hibernating applications.
xdaconfig	A configuration tool that allows the HP-UX root user to adjust the number of communication channels and the number of Data Access servers reserved for high priority requests.

Libraries

These libraries reside in the */opt/TDC_Open/common/lib* directory.

Table 3-6 Library Files

File	Purpose
libcds.sl	A library for Honeywell use only
libhiber.sl liblxs.sl	Libraries used by OpenDDA applications

Modified HP-UX files

These are HP-UX files that are modified.

Table 3-7 Modified HP-UX Files

File	Purpose
/etc/services	Two entries are placed in this file—these entries reserve communications ports for lcn daemon and the cds daemon
/etc/passwd	A user “axm” is placed in this file, with a home directory of /users/axm, which is created during installation
/etc/group	A group “axm” is placed in this file

4. Performance Measurement

4.1 Overview

Introduction

There are four new Processor Status Data Point (PSDP) parameters available as part of the A^xM LCN-side functionality (the node personality). Each of the four PSDP parameters is an array of 100 real-type elements. These four PSDP parameters are:

\$RCVSTA(n)—Provides a set of statistics based on the average number of data transfers (during a 15 second period) from the X-side to the LCN-side

\$RCVCNT(n)—Provides a set of statistics based on the total number of data transfers (during a 15 second period) from the X-side to the LCN-side

\$XMITSTA(n)—Provides a set of statistics based on the average number of data transfers (during a 15 second period) from the LCN-side to the X-side

\$XMITCNT(n)—Provides a set of statistics based on the total number of data transfers (during a 15 second period) from the LCN-side to the X-side

Accessing the parameters from the LCN side

Three methods that can be used to access the parameters are:

Use the AXMPERF schematic (R500 and R600 only)

Use the DATAHNG schematic

Build a custom schematic

Access from CL

Accessing the parameters from the X side

You can access these parameters from the X side using OpenDDA applications or PC Data Exchange (PCDE). PCDE is an X-layer application available from Honeywell that allows you to access LCN point,parameter data from PCs and other devices located on the PIN.

AXMPERF schematic

The following procedure can be used to view the PSDP statistics parameters.

NOTE: This schematic is available only on R500 or later.

Table 4-1 Using the AXMPERF Schematic

Step	Action
1	Press the [SCHEM] button
2	Enter AXMPERF and then press [ENTER]
3	Select "SPECIFY AXM" box and enter number of desired node after \$PRSTS in entry port
4	Press [ENTER] and all implemented values will be displayed (live data)

DATACHNG schematic

The following procedure can be used to view the PSDP statistics parameters.

NOTE: The Toolkit set of schematics (TLK1) must be copied to the HM. The procedure is covered in the *LCN Guidelines* manual.

Table 4-2 Using the DATACHNG Schematic

Step	Action
1	Press the [SCHEM] button
2	Enter DATACHNG and then press [ENTER]
3	Select an empty box on the left and enter the name of the desired parameter in the form: \$PRSTSnn.xx where nn is the node number and xx is the parameter name. For example, to display the second element of the \$RCVSTA(n) array (average number of transmissions) for A*M node 19, enter: \$PRSTS19.\$RCVSTA(2)
4	Press [ENTER] and the value will be displayed (live data)

Custom schematic

The PSDP performance parameters can be used directly in schematics by using the Add Value command and specifying the parameter as shown in the following example:

\$PRSTS19.\$RCVSTA(2)

CL access

You can access the PSDP performance parameters from CL by transferring them to CDS parameters. The following example shows how this can be done.

Programming example

CL V41.11 PERFSTAT

08/01/94 08:54:42:5467

Page

1

```

Line   Loc   Text
1      -----
--
2      -- THIS PROGRAM SHOWS HOW TO ACCESS PROCESSOR STATUS DATA POINT PARAMETERS
3      -- IN AM/CL.  YOU MUST FIRST SET UP A PARAMETER LIST CONTAINING THE
PARAMETERS
4      -- YOU WISH TO MONITOR.  NEXT YOU NEED A CDS TO CREATE A PARAMETER FOR THE
5      -- THE NODE NAME ($PRSTSXX).  USE THE CDS TO ALSO CREATE PARAMETERS FOR EACH
6      -- PERFORMANCE VALUE.
7      -----
8      PACKAGE
9      PARAM_LIST PERFPARM
10     PARAMETER $RCVSTA : NUMBER ARRAY (1..100) --AVERAGE NUMBER OF TRANSMISSIONS
11     END PERFPARM
12
13     CUSTOM
14     PARAMETER NODE_PRF : PERFPARM           -- PROCESSOR STATUS DATA POINT
15     VALUE $PRSTS19                         -- NODE 19 IS AN AM NODE
16
17     PARAMETER R_VALUES : NUMBER ARRAY (1..100)
18     END CUSTOM
19
20     BLOCK PERFSTAT (POINT PERFSTAT; AT GENERAL)
21
22     -- TRANSFER THE PERFORMANCE PARAMETERS TO THE CDS PARAMETERS.  ONCE YOU
HAVE
23     -- THEM IN THE CDS PARAMETERS, THEY BECOME NORMAL POINT.PARAMETERS.
24     -- FOR EXAMPLE, YOU COULD PUT THEM IN A HISTORY GROUP.
25     -- AT A MINIMUM THEY WILL APPEAR TOGETHER ON THE CDS PAGE OF THE POINT.
26     -- THE ASSIGNMENT STATEMENTS BELOW COULD BE REPLACED BY A FOR LOOP.
27
28     6      SET R_VALUES(1) = NODE_PRF.$RCVSTA(1)
29     40     SET R_VALUES(2) = NODE_PRF.$RCVSTA(2)
30     74     SET R_VALUES(3) = NODE_PRF.$RCVSTA(3)
31     108    SET R_VALUES(4) = NODE_PRF.$RCVSTA(4)
32
33     END PERFSTAT
34     END PACKAGE
***** No errors detected

```

4.2 \$RCVSTA(n) Parameter

Discussion

The \$RCVSTA(n) parameter is an array of 100 real-type numbers that provides a set of statistics based on the average number of data transfers (during a 15 second period) from the X-side to the LCN-side.

Table 4-3 Elements of the \$RCVSTA(n) Array

Array Element	Contents
n=1	Average size of messages received by the LCN-side from the X-side during the 15-second sampling period (bytes)
n=2	Average number of messages received by the LCN-side from the X-side during the 15-second sampling period (messages per second)
n=3	Average number of errors on messages received by the LCN-side from the X-side during the 15-second sampling period (errors per second)
n=4..21	Spare elements allocated to communication between the X-side and the LCN-side
n=22..30	Spare elements allocated to data access
n=31..40	Allocated to events
n=41..50	Allocated to file transfer
n=51..70	Reserved for future expansion
n=71	Average number of on-node CDS read requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period (requests per second)
n=72	Average number of off-node CDS read requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period (requests per second)
n=73	Average number of on-node CDS write requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period (requests per second)
n=74	Average number of off-node CDS write requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period (requests per second)

Table 4-3 Elements of the \$RCVSTA(n) Array (Continued)

n=75	Average number of words in on-node CDS read requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period (words per second)
n=76	Average number of words in off-node CDS read requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period (words per second)
n=77	Average number of words in on-node CDS write requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period (words per second)
n=78	Average number of words in off-node CDS write requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period (words per second)
n=79-100	Spare elements allocated to CDS transfer

4.3 \$RCVCNT(n) Parameter

Discussion

The \$RCVCNT(n) parameter is an array of 100 real-type numbers that provides a set of statistics based on the total number of data transfers (during a 15 second period) from the X-side to the LCN-side.

Table 4-4 Elements of the \$RCVCNT(n) Array

Array Element	Contents
n=1	Total size of messages received by the LCN-side from the X-side during the 15-second sampling period (bytes)
n=2	Total number of messages received by the LCN-side from the X-side during the 15-second sampling period
n=3	Total number of errors on messages received by the LCN-side from the X-side during the 15-second sampling period
n=4..21	Spare elements allocated to communication between the X-side and the LCN-side
n=22..30	Spare elements allocated to data access
n=31..40	Allocated to events
n=41..50	Allocated to file transfer
n=51..70	Reserved for future expansion
n=71	Total number of on-node CDS read requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period
n=72	Total number of off-node CDS read requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period
n=73	Total number of on-node CDS write requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period
n=74	Total number of off-node CDS write requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period
n=75	Total number of words in on-node CDS read requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period
n=76	Total number of words in off-node CDS read requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period
n=77	Total number of words in on-node CDS write requests received by the LCN-side from the X-side for locally resident CDS segments during the 15-second sample period
n=78	Total number of words in off-node CDS write requests received by the LCN-side from the X-side for externally resident CDS segments during the 15-second sample period
n=79..100	Spare elements allocated to CDS transfer

4.4 \$XMITSTA(n) Parameter

Discussion

The \$XMITSTA(n) parameter is an array of 100 real-type numbers that provides a set of statistics based on the average number of data transfers (during a 15 second period) from the LCN-side to the X-side.

Table 4-5 Elements of the \$XMITSTA(n) Array

Array Element	Contents
n=1	Average size of messages sent from the LCN-side to the X-side during the 15-second sampling period (bytes)
n=2	Average number of messages sent by the LCN-side to the X-side during the 15-second sampling period (messages per second)
n=3	Average number of errors on messages sent by the LCN-side to the X-side during the 15-second sampling period (errors per second)
n=4..20	Spare elements allocated to communication between the X-side and the LCN-side
n=21	Average number of point.parameters sent by the LCN-side to the X-side and received by the LCN-side from the X-side during the 15-second sampling period (point.parameters per second)
n=22..30	Spare elements allocated to data access
n=31..40	Allocated to events
n=41..50	Allocated to file transfer
n=51..70	Reserved for future expansion
n=71	Average number of host task initiation requests sent from the LCN-side to the X-side during the 15-second sample period (requests per second)
n=72..100	Spare elements allocated to CDS transfer

4.5 \$XMITCNT(n) Parameter

Discussion

The \$XMITCNT(n) parameter is an array of 100 real-type numbers that provides a set of statistics based on the total number of data transfers (during a 15 second period) from the LCN-side to the X-side.

Table 4-6 Elements of the \$XMITCNT(n) Array

Array Element	Contents
n=1	Total size of messages sent from the LCN-side to the X-side during the 15-second sampling period (bytes)
n=2	Total number of messages sent by the LCN-side to the X-side during the 15-second sampling period
n=3	Total number of errors on messages sent by the LCN-side to the X-side during the 15-second sampling period
n=4..20	Spare elements allocated to communication between the X-side and the LCN-side
n=21	Total number of point.parameters sent by the LCN-side to the X-side and received by the LCN-side from the X-side during the 15-second sampling period
n=22..30	Spare elements allocated to data access
n=31..40	Allocated to events
n=41..50	Allocated to file transfer
n=51..70	Reserved for future expansion
n=71	Total number of host task initiation requests sent from the LCN-side to the X-side during the 15-second sample period
n=72..100	Spare elements allocated to CDS transfer

5. Hibernation

5.1 Overview

Attention

This feature requires A^xM and OpenDDA R300.

What is hibernation?

Hibernation is a cooperative functionality involving CL/AM and OpenDDA. Hibernation is available in OpenDDA R300 and later, and requires LCN R433, R531, R500 and R600 or later.

Hibernation allows a CL-initiated OpenDDA application to suspend execution after invocation and initialization, and remain in memory. While in hibernation, CL can periodically activate the task to do some processing and then resume hibernation. CL can also cause the application to gracefully terminate.

What will be covered

This section will provide an overview of hibernation functionality and associated tools. The specifics are covered in the following documents:

CL/AM Reference Manual, Appendix I (AMCL06 Extension)

OpenDDA Reference Manual, R300, Section 6 (OpenDDA Execution Statements).

Advantages of hibernation

CL-initiated X-side applications that are initiated with the call **AMCL06\$Execute_Task_With_Wait** execute until they terminate normally or exit with an error. The CL program uses one A^xM CL queue slot while the X-side application is executing.

By contrast, an application that uses hibernation remains in memory between executions. This offers the following advantages.

Initialization (reading in from disk, resolving external data, and so forth) is performed only once.

Data remains persistent between executions.

The CL does not use an A^xM CL queue slot while the application is hibernating.

Event relationships

The following diagram shows the relationship between CL events and execution of an X-side application that hibernates. It will be useful to refer to this diagram while reading the remainder of this section.

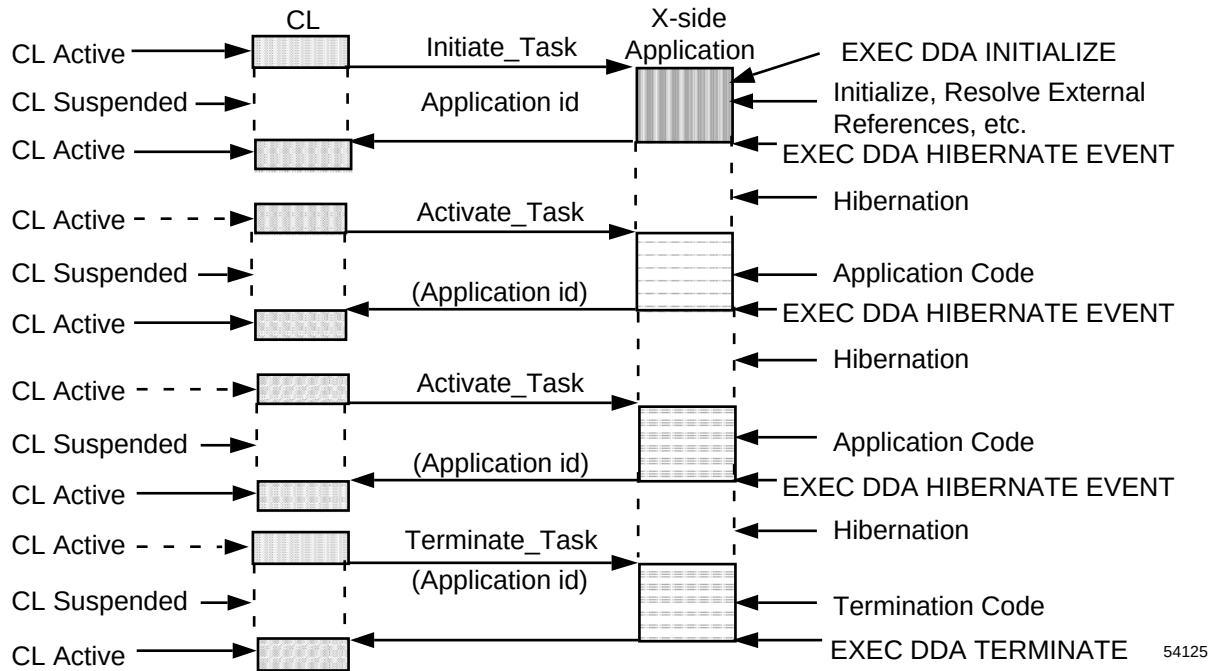


Figure 5-1 Relationship of CL Events and X-side Hibernation

5.2 CL Support

Supporting functions

The following background CL calls are available in the AMCL06_2 set in LCN release R433 and later:

AMCL06\$Initiate_Task—Initiates an OpenDDA application capable of hibernating and receiving background CL events.

AMCL06\$Activate_Task—Sends an activate event to a CL-initiated hibernating OpenDDA application.

AMCL06\$Terminate_Task—Sends a terminate event to a CL-initiated hibernating OpenDDA application, providing a graceful shutdown of the application.

AMCL06\$Get_Hiber_Task_Status—Obtains the current status of a CL-initiated hibernating OpenDDA application capable of receiving background CL events.

Initiate_Task

The background CL subroutine **AMCL06\$Initiate_Task** is used to initiate a new instance of an OpenDDA application. After the call is made, the background CL suspends execution and enters a wait condition until the OpenDDA application has initiated and entered hibernation. The user provides an application name, and once the application has entered hibernation, a unique application id is returned to the CL block. The CL should store this application id in a CDS parameter or other point.parameter and then complete the execution of the CL block. The application id returned from the subroutine is needed to subsequently activate (**AMCL06\$Activate_Task**) and terminate (**AMCL06\$Terminate_Task**) the application.

Syntax of the **AMCL06\$Initiate_Task** subroutine:

```
SUBROUTINE AMCL06$Initiate_Task
(Ret_Status      : OUT NUMBER;      -- Return status of the call
Det_Status       : OUT NUMBER;      -- Detailed return status
Appl_ID          : OUT STRING;      -- Application identifier
Cmd_Line         : IN  STRING;      -- X-side application command line
X_Task_Timeout   : IN  TIME;        -- X-side timeout value
Req_Timeout      : IN  TIME)        -- LCN-side timeout value
```

Refer to Appendix I of the CL/AM Reference Manual for more information about the functionality of the call and for definition of the arguments.

Activate_Task

The background CL subroutine **AMCL06\$Activate_Task** is used to send an activate event to a CL-initiated hibernating OpenDDA application. After the call is made, the background CL suspends execution and enters a wait condition until the OpenDDA application receives and processes the event, presumably executes for a while, and subsequently returns to hibernation. The user provides an application id (obtained from **AMCL06\$Initiate_Task**) and an event string which is passed to the OpenDDA application.

Syntax of the **AMCL06\$Activate_Task** subroutine:

```
SUBROUTINE AMCL06$Activate_Task
  (Ret_Status      : OUT NUMBER;      -- Return status of the call
   Det_Status      : OUT NUMBER;      -- Detailed return status
   Appl_ID         : IN  STRING;      -- Application identifier
   Event_String    : IN  STRING;      -- String passed to application
   X_Task_Timeout  : IN  TIME;        -- X-side timeout value
   Req_Timeout     : IN  TIME)        -- LCN-side timeout value
```

Refer to Appendix I of the CL/AM Reference Manual for more information about the functionality of the call and for definition of the arguments.

Terminate_Task

The background CL subroutine **AMCL06\$Terminate_Task** is used to send a terminate event to a CL-initiated hibernating OpenDDA application. This provides a graceful shutdown of a hibernating OpenDDA application. After the call is made, the background CL suspends execution and enters a wait condition until the OpenDDA application has received and processed the event, and terminated its execution. The user provides an application id (obtained from **AMCL06\$Initiate_Task**) and an event string which is passed to the OpenDDA application.

Syntax of the **AMCL06\$Terminate_Task** subroutine:

```
SUBROUTINE AMCL06$Terminate_Task
  (Ret_Status      : OUT NUMBER;      -- Return status of the call
   Det_Status      : OUT NUMBER;      -- Detailed return status
   Appl_ID         : IN  STRING;      -- Application identifier
   Event_String    : IN  STRING;      -- String passed to application
   X_Task_Timeout  : IN  TIME;        -- X-side timeout value
   Req_Timeout     : IN  TIME)        -- LCN-side timeout value
```

Refer to Appendix I of the CL/AM Reference Manual for more information about the functionality of the call and for definition of the arguments.

**Get_Hiber_Task_
Status**

The background CL subroutine **AMCL06\$Get_Hiber_Task_Status** is used to obtain specific information about a single entry in the X-side hibernating queue. It can be used to obtain information about a known application or to get information about a current hibernating task in the hibernating queue to display on a schematic.

To obtain information about a known application, this subroutine accepts as input an application id (obtained from **AMCL06\$Initiate_Task**) of an OpenDDA hibernating application. It will return information about the hibernating application, including the associated index number in the hibernating queue.

To get information about a current hibernating task in the hibernating queue to display on a schematic, this subroutine accepts as input an index identifier. It will return information about the hibernating application for the given index in the hibernating queue.

The following is the syntax of the **AMCL06\$Get_Hiber_Task_Status** subroutine:

SUBROUTINE AMCL06\$Get_Hiber_Task_Status

Ret_Status	: OUT NUMBER;	-- Return status of the call
Det_Status	: OUT NUMBER;	-- Detailed return status
Task_Status	: OUT NUMBER;	-- hibernating or running
Cmd_Line	: OUT STRING;	-- X-side application command line
Point_Name	: OUT STRING;	-- Point initiating X-side application
Block_Name	: OUT STRING;	-- CL name initiating X-side application
Time_Initiated	: OUT TIME;	-- HP-UX time application initiated
Time_Activated	: OUT TIME;	-- HP-UX time application last activated
Appl_Priority	: OUT NUMBER;	-- current HP-UX priority X_PID
	: OUT NUMBER;	-- X-side process identifier
Appl_ID	: IN OUT STRING;	-- X-side application identifier
Index	: IN OUT NUMBER;	-- Index into X-side hibernating queue
Req_Timeout	: IN TIME)	-- LCN timeout value

Refer to Appendix I of the CL/AM Reference Manual for more information about the functionality of the call and for definition of the arguments.

5.3 OpenDDA Support

Supporting function

The OpenDDA Execution Statement **HIBERNATE EVENT** causes the application to suspend execution awaiting an event initiated from CL. The application suspends execution during execution of the **HIBERNATE EVENT** statement, before the event data and status are returned. Either of the CL calls **AMCL06\$Activate_Task** or **AMCL06\$Terminate_Task** can cause the application to leave hibernation and resume execution. When the application resumes execution, it completes the **HIBERNATE EVENT** statement execution and returns event data and status to the application.

Syntax:

```
EXEC DDA HIBERNATE EVENT,  
                                EVENT_DATA=evt_data,  
                                STATUS=exec_status;
```

evt_data

The variable `evt_data` is a structure that returns information to the application as it exits hibernation as a result of an **AMCL06\$Activate_Task** or **AMCL06\$Terminate_Task** call and resumes execution of code. The information returned is listed in the following table.

Table 5-1 Event Data Structure

Field	Description
Event_type	An integer code that represents the type of the event (ACTIVATE, TERMINATE, or NO_EVENT)
Event_status	The status associated with the event function
Point_name	The entity that initiated the wakeup call
Event_block	Name of the block that activated/terminated the application
Init_block	Name of the block that initiated the application
Event_time	The HP-UX time when the event occurred
Init_time	The LCN time when the application was initiated
Event_string	The CL event string

Reference

For more information about the **EXEC DDA HIBERNATE EVENT** call, refer to Section 6 of the *OpenDDA Reference Manual*.

5.4 X-Side Support Tools

Introduction

Two of the utilities provided by Honeywell in the HP-UX directory `/opt/TDC_Open/common/bin` are used to enable the user to view information about all CL-initiated X-side applications and kill any of these applications—particularly useful for those in hibernation. The two utilities are:

`display_appls`

`kill_appls`

`display_appls`

This is a command line tool used to obtain specific information about all CL-initiated X-side applications. By default, it will display a short version of output, although an option `[-l]` allows display of additional information.

Syntax:

```
display_appls [-l]
```

Applications are listed in two categories:

SYNCHRONOUS APPLICATIONS WITH TERMINATION—Lists tasks initiated by **AMCL06\$Execute_Task_With_Wait**

SYNCHRONOUS APPLICATIONS WITH HIBERNATION—Lists tasks initiated with **AMCL06\$Initiate_Task**

For each task listed, the following information is displayed:

- command line initiating
- point name initiating CL
- block name
- application status (hibernating or running)
- HP-UX process id

When the long version is requested by using the `-l` option, the following is also displayed:

- application id (for hibernating applications only)
- time initiated
- time activated (for hibernating applications only)
- current HP-UX priority

For additional information about this utility, see Appendix I of the *CL/AM Reference Manual*.

kill_appls

The X-side tool `kill_appls` is a command line tool used to abort CL-initiated X-side applications. It can only be executed by a user while a member of the “axm” group. This tool can kill

- one or all CL-initiated X-side applications associated with a point,
- one or all instances of the same application name, or
- applications that are either running or hibernating.

The tool is especially useful for killing X-side applications that can hibernate, because an application that is in hibernation cannot be killed by aborting the CL or inactivating the point to which the CL is attached.

Syntax:

There are two ways to invoke this tool. One way is to specify the application name, and the other way is to specify the associated point.

The syntax when specifying the application name is

```
kill_appls -A application_name {-i application_id | -p process_id | -a}
```

The syntax when specifying the point name is

```
kill_appls -P point_name {-i application_id | -p process_id | -a}
```

One of the following three switches is required:

- I application_id** Application identifier assigned when the application is initiated by the **AMCL06\$Initiate_Task** call.
- p process_id** HP-UX process id.
- a** If used with the **application_name** argument, all executions of the application will be aborted. If used with the **point_name** argument, all X-side applications associated with the point will be aborted.

NOTE: All options are case-sensitive.

For additional information about this utility, see Appendix I of the *CL/AM Reference Manual*.

6. Configuration of Data Access Priority

6.1 Introduction

Attention

This feature requires A^xM release 300.

A^xM services

There are a number of services available in the A^xM. These included Data Access and File Access. Some of these services can take very long periods of time to complete an associated action. To ensure that resources are available for high priority control actions, Honeywell has added functionality in A^xM release 300 that allows the user to configure Data Access priority.

Application class

There are two basic classes of applications:

High priority control applications.

Low priority information level applications

High priority control applications

These control applications often have stringent timing requirements for reading and writing LCN data. They are involved in closed loop control strategies and often, but not always, transfer smaller amounts of data. Therefore, they need access to the LCN for bringing in reasonable amounts of data, but the "path to the valve" must be available when needed.

Low priority information level applications

These applications typically have more relaxed requirements for access to the LCN for reading/writing of data. They tend to bring in a large amounts of data for functions such as data collection (TPH), PC desktop viewing (PCDE), or background types of applications. Because these are not control applications, they can often tolerate a little more variability in their access to the LCN, and often do not write any data back to the LCN. These applications can therefore take a great deal of time to complete actions and use a great deal of channels and memory in the AM-side of the A^xM to serve the large data access requests.

Configuration goals

The configuration functionality is provided to allow the user to “tune” certain configuration settings to match up with the mix of application types executing in an A^xM. The following are the ultimate goals:

Assure a clean "path to the valve" for the control applications coexisting in the same A^xM with applications used for gathering data for information applications.

Do not allow any type of platform service that can take very long period of time (like File Services) to be able to block control application requests.

Provide sufficient low priority resources so that low priority requests are not blocked out indefinitely.

Resources

There are two resources that the user can manage:

Data access channels

Data access servers

Data access channels

There are only a limited number of channels available for interfacing between the AM-side and X-side. You must assure that a portion of these resources are set aside for use by control applications when needed.

In an A^xM with control applications coexisting with a large number of noncontrol applications, the potential exists for the noncontrol applications to consume all the available channels, use them for long time period tasks, and therefore lock out control applications from running within an allowable, deterministic, and repeatable time period.

To ensure that this will not happen, A^xM R300 will allow the user to configure the A^xM to set aside a defined number of the data access channels for high priority actions only. Then the only contention for these channels will be between high priority control applications (such as OpenDDA programs running at Control and RCasEnb data priority levels). Low priority tasks will then have to share the remaining channels with other noncontrol applications (for example, PCDE, OpenDDA programs running at NoControl Data Priority levels, TPH collectors, and programs using platform file services).

Data access servers

The second resource to manage is the data access servers (and associated AM-side pool memory used during the execution of these requests). Even when applications are prioritized as high or low for determination of access to data access channels, we must then be concerned about priority with multiple queued requests against the data access servers. To support this, A^xM R300 will allow the user to configure the AM-side to set aside a defined number of data access servers to service high priority requests only.

In addition, requests from the X-side can make data access requests with an associated priority. The result is that even within the scope of high/low classes, there is the capability to allow priorities within these classes. For example, OpenDDA NoControl applications have a higher priority within the low class of requests than, for example, an application such as PCDE. Therefore, if a large number of requests are being queued for data access, the servers will take care of the OpenDDA applications first. This allows for some level of "jumping to the front of the line" within the priority classes.

Summary

A^xM R300 allows a user to customize an A^xM's data access priority. The user can ensure that a defined number of channels, data access servers, and corresponding pool memory, have been set aside to service high priority requests. The result is that after a node has demonstrated a defined operational characteristic for various load configurations, the user can have a good sense of assurance that this will be repeatable behavior. This will be the case even if, for example, a large number of PC desktop users should decide to call up Excel spreadsheets that use PCDE for access to LCN data, and thereby demand a great deal of A^xM resources.

6.2 xdaconfig Configuration Tool

Files

The configuration of the priority of the Data Access paths through the X-layer are defined in a configuration file **xdapriority.cfg** that is located in **/etc/opt/TDC_Open/common**. The configuration file defines the number of channels reserved for high priority requests as opposed to low priority requests. The file also defines the number of Data Access servers assigned to high priority requests as opposed to low priority requests. As shipped from Honeywell in the A^xM, the **xdapriority.cfg** file is set to four high priority DA servers and 10 high priority channels. For U^xS, the file is set to zero for both high priority servers and channels.

Configuration tool

A configuration tool is provided to adjust these two priority related configuration items. The tool, **xdaconfig**, is in the directory **/opt/TDC_Open/common/bin**. The tool is a command line tool that asks the user for the number of channels reserved for high priority requests (0 to 20) and for the number of Data Access servers reserved for high priority requests (0 to 4). The remaining channels and Data Access servers are then assigned to low priority.

Discussion

For security, the configuration file is not a text file. In addition, the configuration file and the configuration tool have restricted access to the system administrators only (root level).

Low priority requests **cannot** be serviced by a high priority queue or by a high priority Data Access serve. High priority requests, however, **can** be serviced by a low priority queue and servers. Therefore, if no DA servers are configured for high priority, setting the number of high priority channels to the minimum (0) and the number of Data Access servers to a minimum (0) effectively sets all the DA servers to service both high and low priority requests with the requests prioritized within a single queue.

Attention

Whenever the configuration file is changed, the LCN personality must be reloaded or the entire A^xM must be restarted in order for the changes to take effect.

Sample session

The following is a sample session utilizing the Data Access configuration tool.

```
$ cd /opt/TDC_Open/common/bin
```

```
$ xdaconfig
```

```
***      THE CURRENT DATA ACCESS CONFIGURATION ARE ***
```

```
Number of DA servers for HIGH          = 4
```

```
Number of channels reserved for HIGH    = 10
```

```
Do you want to update the configuration file? (Y or N) Y
```

```
Please enter the number of DA servers for 'HIGH' priority (0 to 4 or N) 3
```

```
Please enter the number of channels reserved for 'HIGH' priority (0 to 20 or N) 8
```

```
---      THE UPDATED DATA ACCESS CONFIGURATION ARE ---
```

```
Number of DA servers for HIGH          = 3
```

```
Number of channels reserved for HIGH    = 8
```

```
DA CONFIGURATION COMPLETE
```

```
$
```


Index

\$

\$SRCVCNT(n), 39, 44
\$RCVSTA(n), 39, 42
\$XACCESS, 30
\$XMITCNT(n), 39, 46
\$XMITSTA(n), 39, 45

A

about audit files., 26
AMCL06, 30
AMCL06\$Activate_Task, 49, 50
AMCL06\$Get_Hiber_Task_Status, 49, 51
AMCL06\$Initiate_Task, 49
AMCL06\$Terminate_Task, 49, 50
AMCL06_2, 19, 20, 21, 28
Application ModuleX Architecture
 Chassis types, 16
 Coprocessor hardware, 18
 Coprocessor software, 18
 Five-Slot Module configuration, 16
 Function of components, 17
 Ten-Slot Module configuration, 16
axm group, 29
AXMPERF schematic, 39

C

chacl, 25
chgrp, 25
chmod, 25
chown, 25
Components of the A^XM, 17
configuration files, 35
Configurations
 Expanded network configuration, 22
 Minimum network configuration with MP-AMXST1
 workstation, 21
 Minimum network configuration with U^XS, 20
 Standalone coprocessor, 19
 Where to find more information, 24
control domain, 14

D

Data Access Priority, 55
 Introduction, 55
 xdaconfig, 58
Data Definition and Access, 14
DATACHNG schematic, 39, 40
DDA, 14
display_appls, 53

F

File system
 Configuration files discussion, 35

Data files, 36
Discussion, 33
Executables, 36
Modified HP-UX files, 38
Reference configuration files, 35
Summary of key directories, 34
Working configuration files, 36
Function of A^XM components, 17

H

Hibernation, 47
 Advantages, 47
 CL Support, 49
 OpenDDA Support, 52
HP-UX, 18

I

information domain, 14

K

kill_appls, 54

O

OpenDDA, 14
OpenUSE, 14

P

PA-RISC, 18
Performance measurement
 Accessing the parameters from the LCN side, 39
 AXMPERF schematic, 40
 Custom schematic, 40
 DATACHNG schematic, 40
 Introduction, 39
 Programming example, 41
Plant Information Network, 14
Processor Status Data Point, 39
PSDP, 39

Index

S

security, 25

Security

Physical, 27

Procedural, 27

Security—HPUX

Audit Files, 26

File permissions, 25

important categories, 25

Introduction, 25

Network security, 26

User security, 26

Security—LCN

A^XM failover to AM, 28

Directory for CL-initiated applications, 29

Function of XACCES, 31

global X-access switch, 30

Overview, 28

Topics to be covered, 28

Using test data with OpenDDA, 29

T

TotalPlant, v

W

WSI2 board, 18

X

X Windows environment, 15

XACCES external load module, 31

X-access switch, 30

xdaconfig, 58

Sample session, 59

xdapriority.cfg, 58

FAX Transmittal

FAX No.: (602) 313-4842

TO: _____

Total FAX pages: _____
(including this page)

Reader Comments

Title of Document: **Application Module^x User Guide**

Document Number: **AX09-300**

Issue Date: **12/98**

Comments: _____

Recommendations: _____

FROM:

Name: _____ **Date:** _____

Title: _____

Company: _____

Address: _____

City: _____ **State:** _____ **ZIP:** _____

Telephone: _____ **FAX:** _____

FOR ADDITIONAL ASSISTANCE:

Write	Call
Honeywell Inc. Industrial Automation and Control Automation College 2820 West Kelton Lane Phoenix, AZ 85053-3028	Technical Assistance Center (TAC) 1-800-822-7673 (48 contiguous states except Arizona) 602-313-5558 (Arizona)

Honeywell

Industrial Automation and Control

Automation College
2820 W. Kelton Lane
Phoenix, AZ 85053-3028